

Dénormalisation avec PostgreSQL : Vues, Tableaux, Types composites

3.1 Vues

1. Connaître la moyenne de chacune des évaluations est une information utile pour les commissions de passage ainsi que pour l'édition des bulletins. On souhaite que cette moyenne soit normalisée sur 20. Proposer une requête SQL qui affiche la table `evaluation` avec comme colonne supplémentaire la moyenne de l'évaluation sur 20.
2. Cette moyenne est si souvent utile que nous voudrions y avoir accès facilement sans avoir à effectuer de requête. Peut-on ajouter cette colonne à la table `evaluation` en utilisant une colonne générée ?
3. Créer une vue `vue_evaluation` avec cette colonne supplémentaire.
4. Interprétez le plan d'exécution de la commande affichant le contenu de cette vue :

```
EXPLAIN ANALYSE SELECT * FROM vue_evaluation ;
```

5. Créer une vue matérialisée `vuemat_evaluation` avec cette colonne supplémentaire.
6. Interprétez le plan d'exécution de la commande affichant le contenu de cette vue matérialisée :

```
EXPLAIN ANALYSE SELECT * FROM vuemat_evaluation ;
```

La commande est-elle plus rapide que sur la vue classique ? D'un facteur de combien ?

7. Intéressons-nous à la requête affichant les informations concernant les évaluations du module 2 de la vue `vue_evaluation` :

```
SELECT * FROM vue_evaluation WHERE id_module = 2 ;
```

Le SGBD doit-il recalculer entièrement la vue `vue_evaluation` pour répondre à cette requête ? Cette requête est-elle plus lente ou plus rapide que celle exécutée plus haut question 4 consistant à afficher tout le contenu de la vue. Comment observer ce que le SGBD fait réellement ?

8. Que dire de cette même requête sur la vue matérialisée :

```
SELECT * FROM vuemat_evaluation WHERE id_module = 2 ;
```

3.2 Tableaux

9. Écrire une requête qui donne le nombre d'évaluations de chacun des modules.

id_module	intitule	Nb. eval.
11	Gestion de projet & des organisations	2
9	Bases de la communication	7
...	...	...

10. En partant de la requête précédente, écrire une requête qui donne, sous forme de tableau, la liste des identifiants des évaluations de chacun des modules. La fonction d'agrégation `array_agg()` est parfaitement adaptée à cette situation.

id_module	intitule	Tab. eval.
11	Gestion de projet & des organisations	{33653,33654}
9	Bases de la communication	{33055,33066,33519,...}

- Modifier la requête précédente pour avoir, non plus un tableau d'identifiants, mais un tableau de couples (identifiant,nom) (i.e. un tableau à deux dimensions). Comme un tableau ne contient que des éléments d'un même type, il faudra transtyper les identifiants en chaîne de caractères.
- Écrire une requête qui affiche pour chaque évaluation les notes maximum et minimum sur 20.

id_evaluation	nom	id_module	max	min
32831	Minicontrol 4 - Exercice 2	12	20	0.4
33640	Phase complémentaire	4	16.49	5.24

- Réécrire la requête de la question 12, en utilisant la requête de la question 11 en tant que CTE, pour ajouter les colonnes des notes maximum et minimum à chacune des évaluations.
- Faire de la requête précédente une vue. Penser à nommer la dernière colonne. Les notes doivent être arrondies à un chiffre après la virgule (utiliser la fonction `ROUND(<val>::numeric,1)`).
- En utilisant cette vue, écrire une requête qui affiche pour chaque module le nom de la première évaluation et sa note maximale.

3.3 Types composites (ou UDT pour *User Defined Types*)

Dans la vue de la question 14, la dernière colonne contient des tableaux à deux dimensions. Les lignes d'un tableau représentent l'ensemble des évaluations, et les colonnes permettent de représenter respectivement l'identifiant, le nom, la note maximum et la note minimum de chacune des évaluations. Ces quatre colonnes sont identifiées par leur indice (de 1 à 4), ce qui n'est pas très explicite quand à leur contenu. Nous allons remplacer ce tableau à deux dimensions par un tableau à une dimension d'élément de type composite (UDT).

- Créer un type composite pour représenter un identifiant, un nom, une note maximum et une note minimum.
- Supprimer la vue de la question 14.
- Recréer une vue équivalente à celle de la question 14 avec des tableaux unidimensionnels constitués d'éléments du type déclaré dans la question 16.
- En utilisant cette vue, écrire une requête qui affiche pour chaque module le nom de la première évaluation et sa note maximale.